Dijkstra's Algorithm :-

Dijkstra's algorithm, named after Dutch computer scientist Edsger W. Dijkstra, is a fundamental algorith-m used in graph theory and computer science for finding the shortest path between nodes in a graph theory with non-negative edge weights. Orignally designed to solve the single-source shortest path problem, Dijkstra's algorithm efficiently determines the shortest path from a designated starting node to all other nodes in the graph.

Application :- Network Routing protocols, transportation networks and data processing system.

Single-source shortest-path problem :-

It is defined as follows :-

For a given vertex called the source in a weighted connected graph, the problem is to find shortest paths to all its other vertices.

The single-source shortest-paths problem asks for a family of paths, each leading from the source to a different vertex in the graph, though some paths may, of course, have edges in common.

(Algorithm is applicable to undirected & directed graphs with non-negative weights only)

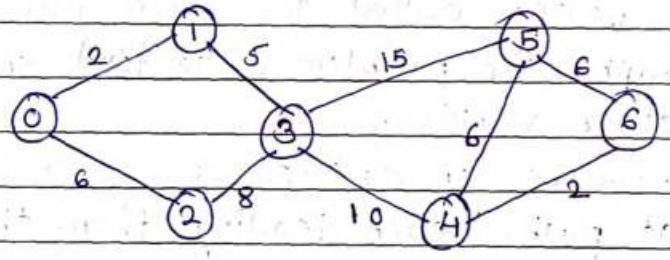
Working :-

The algorith works by maintaining a set of vertices that have already been visitied, and a set of vertices that have not yet been visited. The algorithm starts at the source vertex and adds it to the set of visited vertices. Then it repeatedly finds the vertex in the set of unvisited vertices that has the shortest known path to the source vertex, and adds that vertex to the set of visited vertices. This process continues until all vertices have been visited.

The Dijkstra algorithm is a greedy algorithm, which means that it makes the best possible choice at each step, without considering the future consequences of its decision. In the case of Dijkstra algorithm, the best possible choice is to add the vertex with the shortest known path to the source vertex to the set of visited vertices visited.

The Dijkstra algorithm is a very efficient algorithm for finding the shortest paths in a graph. Its time complexity is $O(V \log V)$, where V is the number of vertices in the graph.

Example



This algorithm will generate the shortest path from 0 to all the other nodes in the graph.

Assume → weight of the edges ~~are~~ is the distance between two nodes.

We will have the shortest path from node 0 → 1 from node 0 → 2, From node 0 → 3 & so on for every node in the graph.

- The distance from the source node to itself is 0
- The distance from the source node to all other nodes has not been determined yet, so we use the ∞ to represent it initially.

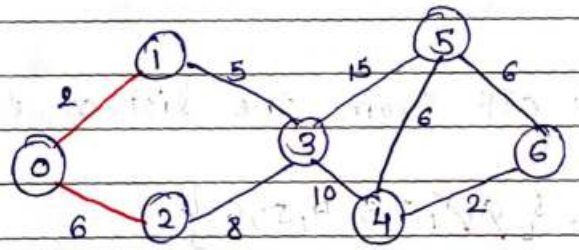
	Distance
0	0
1	∞
2	∞
3	∞
4	∞
5	∞
6	∞

Unvisited Nodes
{ 0, 1, 2, 3, 4, 5, 6 }

Since we are choosing to start at node 0, we can mark this node as visited.
Or we cross it off from the list of unvisited nodes and add a red border.

Unvisited Nodes: { ~~0~~, 1, 2, 3, 4, 5, 6 }

Now we need to start checking the distance from node 0 to its adjacent nodes. As you can see, these are nodes 1 and 2 (Red edges)



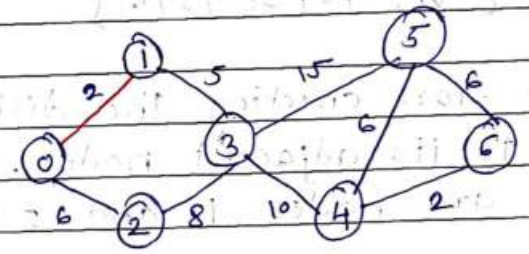
We need to check if we have found the shortest path to reach it. We are simply making an initial examination process to see the options available. We need to update the distances from node 0 → 1 & node 2 with the weights of the weights of the edges that connect them to node 0 (the source node). These weights are: 2 and 6, respectively.

	Distance	
0	0	
1	~~∞~~	2
2	~~∞~~	6
3	∞	
4	∞	
5	∞	
6	∞	

After updating the distances of the adjacent nodes, we need to:

- Select the node that is closest to the source node based on the current known distances.
- Mark it as visited.
- Add it to the path.

If we check the list of distances, we can see that node 1 has the shortest distance to the source node (a distance of 2), so we add it to the path.



Visited nodes shown with red edges.

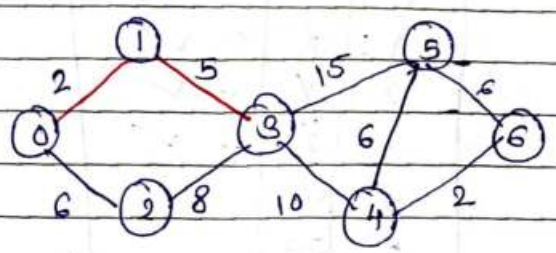
We cross it off from the list of unvisited nodes

Unvisited Nodes : { ~~0~~, ~~1~~, 2, 3, 4, 5, 6 }

Now we need to analyze the new adjacent nodes to find the shortest path to reach them. We will only analyze the nodes that are adjacent to the nodes that are already part of the shortest path (the path marked with red edges).

Node 3 & 2 are both adjacent to nodes that are already in the path because they are already directly connected to node 1 & node 0, respectively. So we will analyze these nodes in the next step. Since we already have the distance from the source node to node-2 written down in our list we don't need to update the distance this time. We only need to update the distance from the source node to the new adjacent node. i.e. node-3

0	0	
1	~~∞~~	2
2	~~∞~~	6
3	~~∞~~	7
4	∞	
5	∞	
6	∞	

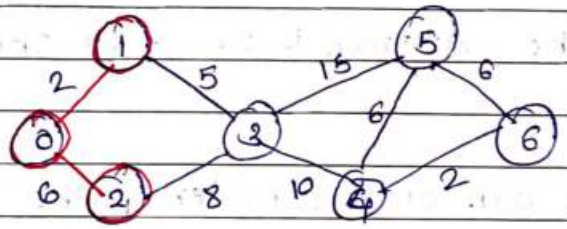


$0 \rightarrow 1 \rightarrow 3 = 7$ (total distance)

Now that we have the distance to the adjacent nodes, we have to choose which nodes, we have to choose which node will be added to the path. We must select the unvisited node with the shortest (currently known) distance to the source node.

From the list of distances, we can immediately detect that this is node 2 with distance 6:

We add it to the path graphically with a red border around the node & a red edge.



Visited nodes $\{2, 6\}$

Unvisited Nodes:

$\{\cancel{0}, \cancel{1}, \cancel{2}, 3, 4, 5, 6\}$

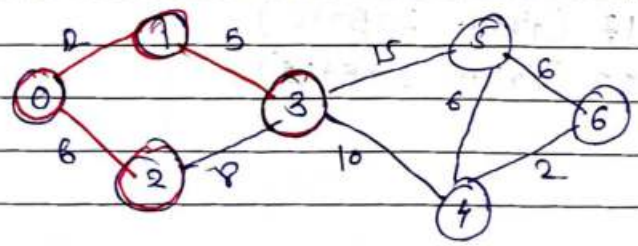
Now that we have the need to repeat the process to find the shortest path from the source node to the new adjacent node, which is node 3.

Two possible paths $0 \rightarrow 1 \rightarrow 3$ or $0 \rightarrow 2 \rightarrow 3$

Lets decide which is shortest path.

$0 \rightarrow 1 \rightarrow 3$ distance = 7 ✓ shortest

$0 \rightarrow 2 \rightarrow 3$ distance = 14



Distance.

0 :	0	
1 :	~~∞~~	2 ✓
2 :	~~∞~~	6 ✓
3 :	~~∞~~	7 ✓
4 :	∞	
5 :	∞	
6 :	∞	

Unvisited Nodes:
{ ~~0~~, ~~1~~, ~~2~~, ~~3~~, 4, 5, 6 }

Now we will check the new adjacent nodes tha
we have not visited so far.
This time, these nodes are node 4 & node 5 sin
they are adjacent to node 3.

We update the distances of these nodes to the sour
node, always trying to find a shorter path, if
possible.

For node 4: the distance is 17 from the path
$0 \rightarrow 1 \rightarrow 3 \rightarrow 4$

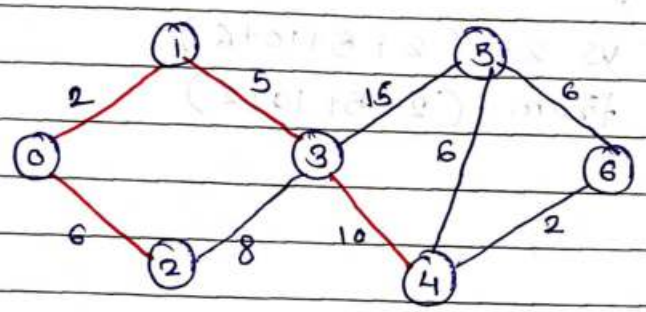
For node 5: the distance is 22 from the path
$0 \rightarrow 1 \rightarrow 3 \rightarrow 5$

* Notice that we can only consider paths that wil
take us through edges that have not been added
to the shortest path (for ex- we cannot form a
path that goes through the edge $2 \rightarrow 3$).

Distance

0 : 0
1 : ~~∞~~ 2 ✓
2 : ~~∞~~ 6 ✓
3 : ~~∞~~ 7 ✓
4 : ~~∞~~ 17 (from 2+5+10)
5 : ~~∞~~ 22 (from 2+5+15)
6 : ∞

We need to choose which unvisited node will be marked as visited now. In this case, it's node 4 because it has the shortest distance in the list of distances. We add it graphically in the diagram:



Now we will mark it as visited

Distance :

0	: 0	
1	: ∞	2 ✓
2	: ∞	6 ✓
3	: ∞	7 ✓
4	: ∞	17 ✓
5	: ∞	22 ✗
6	: ∞	

Unvisited Nodes:
{ ~~0~~, ~~1~~, ~~2~~, ~~3~~, ~~4~~, 5, 6 }

For node 5 :

The 1st option is to follow the path 0→1→3→5, which has a distance of 22 from the source node (2+5+15).

This distance was already recorded in the list of distances in a previous step.

The 2nd option would be to follow the path 0→1→3→4→5, which has a distance of 23 from the source node (2+5+10+6)

1st path is shorter, so we choose it for node 5.

For node 6

0→1→3→4→6 distance = 19 from the source node (2+5+10+2)

Distance

0 : ∞

1 : ~~∞~~ 2 ✓

2 : ~~∞~~ 6 ✓

3 : ~~∞~~ 7 ✓

4 : ~~∞~~ 17 ✓

5 : ~~∞~~ 22 vs 23 (2+5+10+6)

6 : ~~∞~~ 19 from (2+5+10+2)

Distance

0 : 0

1 : ∞ 2 ✓

2 : ∞ 6 ✓

3 : ∞ 7 ✓

4 : ∞ 17 ✓

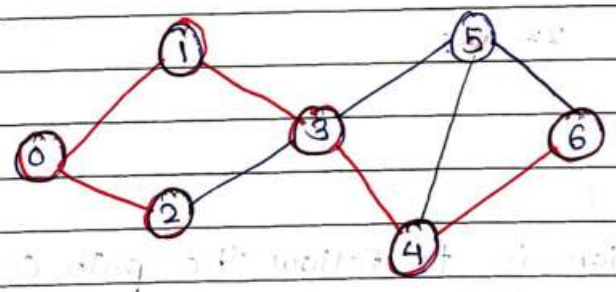
5 : ∞ 22

6 : ∞ 19 ✓

Unvisited Nodes :

{ ~~0~~, ~~1~~, ~~2~~, ~~3~~, ~~4~~, 5, ~~6~~ }

So we have this path (marked in red)



Only one node has not been visited yet, nodes
Lets see how we can include it in the path

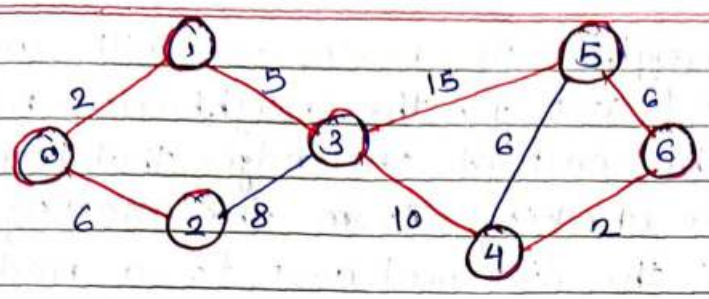
There are 3 different paths that we can take to reach node 5 from the nodes that have been added to the path:

1 : 0 → 1 → 3 → 5 = 22 (2+5+15)

2 : 0 → 1 → 3 → 4 → 5 = 23 (2+5+10+6)

3 : 0 → 1 → 3 → 4 → 6 → 5 = 25 (2+5+10+2+6)

We select shortest path (option 1)



Distance

0 : 0

1 : ~~∞~~ 2 ✓

2 : ~~∞~~ 6 ✓

3 : ~~∞~~ 7 ✓

4 : ~~∞~~ 17 ✓

5 : ~~∞~~ 22 ✓

6 : ~~∞~~ 19 ✓

Unvisited Nodes :

{ ~~0~~, ~~1~~, ~~2~~, ~~3~~, ~~4~~, ~~5~~, ~~6~~ }

Final path

0 → 1 → 3 → 4 → 6

Rules for running Dijkstra's Algorithm

1. From the starting node, visit the vertex with the smallest known distance/cost.
2. Once we've moved to the smallest-cost vertex, check each of its neighboring nodes.
3. Calculate the distance/cost for the neigboring nodes by summing the cost of the edges leading from the start vertex.
4. If the distance/cost to a vertex we are checking is less than a known distance, update the shortest distance for that vertex.

Steps

1. Everytime that we set out to visit a new node, we will choose the node with the smallest known distance/cost to visit first.
2. Once we've moved to the node we are going to visit, we will check each of its neighboring nodes.

3. For each neighboring node, we will calculate the distance / cost for the neighboring nodes by summing the cost of the edges that lead to the node we've checking from the starting vertex

4. Finally, if the distance/ cost to a node is less than a known distance, we'll update the shortest distance that we have on file for that vertex.

Minimum cost spanning trees:

A spanning tree of a connected graph is its connected acyclic subgraph (i.e. a tree) that contains all the vertices of the graph. A minimum spanning tree of a weighted connected graph is its spanning tree of the smallest weight, where the weight of the tree is defined as the sum of the weights on all its edges.

The minimum spanning tree problem is the problem of finding a minimum spanning tree for a given weighted connected graph.

Algorithms to find minimum spanning tree

Kruskal | Prim

Kruskal's & Prim's algorithms are greedy algorithms.

Kruskal's algorithm:-

Kruskal's algorithm is a classic algorithm used in graph theory to find the minimum spanning Tree (MST) of a connected, undirected graph. The MST is a subset of the edges that connects all the vertices without any cycles & with the minimum possible total edge weight. Kruskal's algorithm is greedy, meaning it builds the MST by always choosing the next shortest edge that doesn't form a cycle.

Steps to Kruskal's algorithm

1. Sort All edges
2. Initialize subsets
3. Iterate over sorted Edges
4. Repeat Until MST is complete.

Importance of Kruskal's Algorithm in Graph Theory

Kruskal's Algorithm is one of the cornerstone algorithms in graph theory, widely used for finding the Minimum Spanning Tree (MST) of a connected, weighted graph.

1. Efficient MST Construction

Kruskal's algorithm efficiently identifies the MST, which minimizes the total weight of edges in a graph while ensuring all vertices are connected without forming cycles. This is crucial

Knapsack Problem (Greedy Approach)

The knapsack problem is defined from the maximization problem mentioned below.

Given a bag, with maximum weight capacity of W and a set of items, each having a weight and a value associated with it. Decide the no. of each item to take in a collection such that the total weight is less than the capacity and the total value is maximized.

Example : Knapsack Problem (Fractional)

Objects	Obj 1	Obj 2	Obj 3
Profit	25	24	15
Weight	18	15	10

Knapsack Capacity (M) = 20

Algo Method (3)

→ for i=1 to n
O(n) Calculate Profit/weight
→ Sort objects in decreasing
O(logn) order of P/W ratio.
→ for i=1 to n
if M>0 and $W_i \leq M$
$M = M - W_i$;
$P = P + P_i$;
else break,
O(n) if (M>0)
$P = P + P_i\left(\frac{M}{W_i}\right)$

$O(n) + O(n\log n) + O(n)$
$= O(n\log n)$

(1) Greedy about Profit

Prefer objects with more profit

Obj1 18

$25 + \frac{2}{15} \times 24$

$= 25 + 3.2$

$= \underline{\underline{28.2}}$

(2) Greedy about weight

Prefer objects with less weight

10 Obj3

$15 + \frac{10}{15} \times 24$

$= 15 + \frac{240}{15} = \underline{\underline{31}}$

Greedy about Both :

	Obj1	Obj2	Obj3
(3) P/W	$\frac{25}{18}$	$\frac{24}{15}$	$\frac{15}{10}$
	1.3	1.6	1.5

weight

5

15 Obj2

24 Profit

$24 + \frac{5}{10} \times 15 = 24 + 7.5$

$= 31.5$

Types of Knapsack Problem

0/1 Knapsack Problem	Fractional knapsack problem
↳ Items are either completely or no items are filled in a knapsack.	↳ We can divide the item.
↳ Solved by using dynamic programming approach	↳ Solved by using greedy approach.
↳ 0/1 knapsack problem has an optimal structure	↳ The fractional knapsack problem also has an optimal structure
↳ In the 0/1 knapsack problem, we are not allowed to break items.	↳ Fractional knapsack problem, we can break items for maximizing the total value of the knapsack.
↳ 0/1 knapsack problem, finds a most valuable subset item with a total value less than equal to weight.	↳ In the fractional knapsack problem, find a most valuable subset item with a total value equal to weight.
↳ In the 0/1 knapsack problem we can take objects in an integer value.	↳ In the fractional knapsack problem, we can take objects in fractions in floating points.
↳ The 0/1 knapsack doesnot have greedy choice property	↳ The fraction knapsack do have greedy choice property.

Job Sequencing with deadlines (Greedy Approach)

Job sequencing with deadlines is a problem that involves scheduling a set of jobs to maximize profit while adhering to their respective deadlines. This approach assumes that each job can be completed in exactly one unit of time.
If jobs have different durations, a more advanced scheduling algorithm might be necessary.

Job Sequencing

	J_1	J_2	J_3	J_4
Profit	50	15	10	25
Deadline	2	1	2	1

- Uniprocessor
- No-Premption
- 1 unit time per each job

(1) Job

J_1	J_3

0 1 2

Total Profit

Profit 50 + 10 60

(2)

J_4	J_1

0 1 2

Total profit

Profit 25 + 50 75

Algorithm :

(nlogn) 1) Arrange All jobs in decreasing order of Profit
2) For each Job (mi), do linear search to find particular slot in array of size (n),
where n = maximum deadline
m = total Jobs.

$O(n^2)$

Find an optimal solution to the knapsack instances

$n = 7$, $m = 15$

Profit = {10 , 5 , 15 , 7 , 6 , 18 , 3}

Weight = {2 , 3 , 5 , 7 , 1 , 4 , 1}

Given :

$n =$

Step1

	1	2	3	4	5	6	7
Profit	10	5	15	7	6	18	3
weight	2	3	5	7	1	4	1
P_i/w_i	P_1/w_1 5	P_2/w_2 1.67	P_3/w_3 3	P_4/w_4 1	P_5/w_5 6	P_6/w_6 4.5	P_7/w_7 3
	2 ✓	6	4 ✓	7	1 ✓	3 ✓	5

Step2 : Arrange this profit/weight ratio in non-increasing order as n values) Since the highest P/W ratio is 6 . That is P_5/W_5 , so 1st value is 5 2nd highest P/W ratio is 5 i.e P_1/W_1 so 2nd value is 1. Similarly , calculate such n values and arrang them in non-increasing order.

Order = (5, 1, 6, 3, 7, 2, 4)

Step3 : To find optimal solution using $m = 15$ & $n = 7$

Consider $x_5 = 1$, Profit = 6

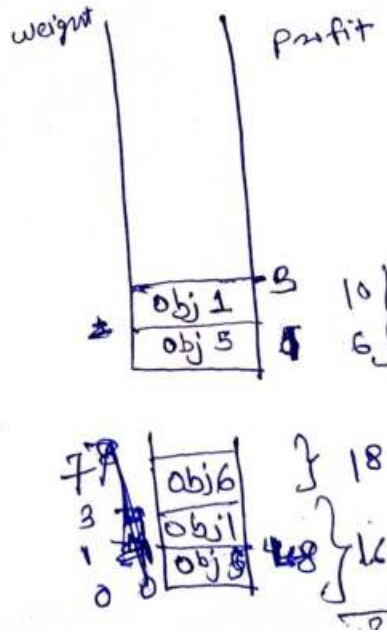
Then . consider $x_1 = 1$, Profit = 10

So weight upti1 now = 1+2 = 3

Now $x_6 = 1$, Profit = 18

so total profit = 16+18 = 34

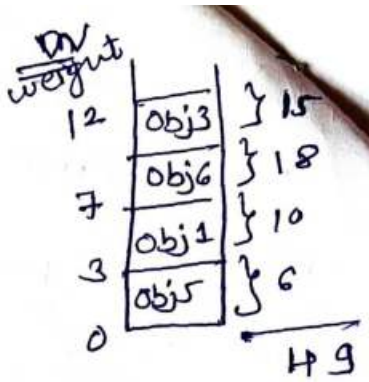
And weight upti1 now = 3+4 = 7



Now $x_3 = 1$, profit $= 15$

So total profit $= 34 + 15 = 49$

And weight upto now $= 7 + 5 = 12$



Now $x_7 = 1$, profit $= 3$

So total profit $= 49 + 3 = 52$

And weight upto now $= 12 + 1 = 13$

Since $m = 15$, so we required only 2 units more.

Therefore $x_2 = 2/3$

So total profit $= 52 + 5 \times 2/3$

$= 52 + 3.33$

$= 55.3$

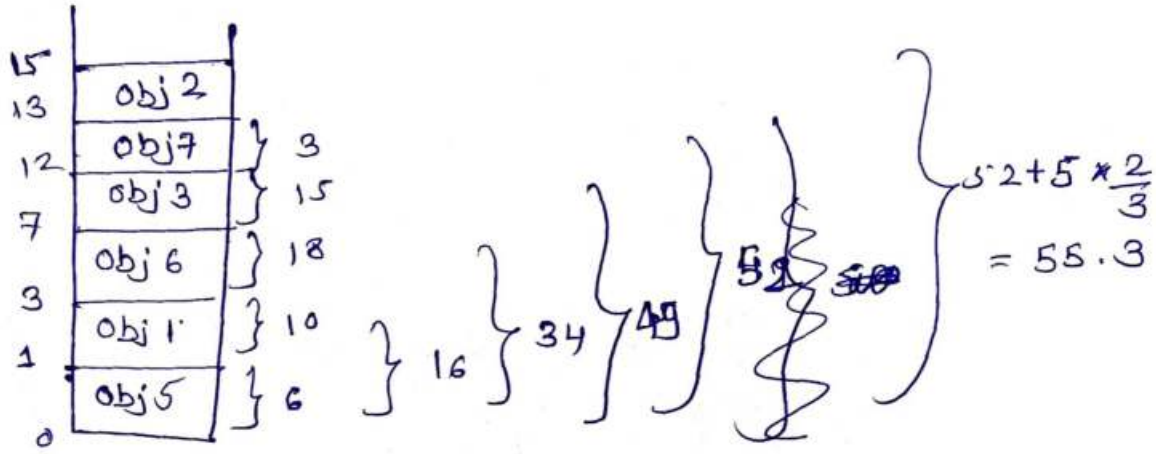
And weight upto now $= 13 + 3 \times 2/3 = 15$

Thus, the optimal solution that gives maximum profit is,

$(1, 2/3, 1, 0, 1, 1, 1)$

	obj 5	obj 1	obj 6	obj 3	obj 7	obj 2	obj 4
profit	6	10	18	15	3	5	7
weight	1	2	4	5	1	3	7

weight



we need more 2 units

weight = $13 + 3 \times 2/3 = 15$

Thus, the optimal solution that gives max^m profit is $(1, 2/3, 1, 0, 1, 1, 1)$

2. Obtain the solution to following fractional greedy knapsack problem where $n=5$, $m=100$, $(P_1, P_2, \cdots P_5) = (10, 20, 30, 40, 50)$ and $(W_1, W_2, \cdots W_5) = (20, 30, 66, 40, 66)$

~~W_1 W_2 W_3 W_4 W_5~~

~~W 20 30 66 40 66~~

order = {4, 5, 2, 3, 1}

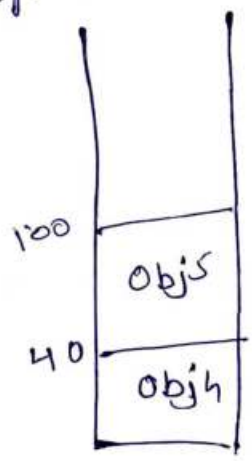
	obj1	obj2	obj3	obj4	obj5
P	10	20	30	40	50
W	20	30	66	40	66
P/W	0.5	0.66	0.5	1	0.75
	5	3	4	1	2

	obj4	obj5	obj2	obj3	obj1
P	40	50	20	30	10
W	40	66	30	66	20

{4, 5, 2, 3, 1}

weight

m = 100
profit



40

$40 + 50 \times \frac{60}{66} = 40 + 50 \times 0.90$

$= 40 + \cancel{27.27}\ 45.45$

$= \underline{85.45}$

weight $= 40 + \cancel{60} \times \frac{60}{\cancel{66}} = 40 + 60$

$= \underline{100}$